Review Star Rater

Fine-tuning Qwen2.5 with QLoRA to predict Yelp star ratings: an experiment report

Hugging Face: shivkrchy9876 · GitHub: Shiva123-del · October 2026 · Kaggle T4 GPU · github.com/Shiva123-del/review-star-rater-qlora

42.7% → 65.0%
exact-star accuracy (base 0.5B → best model)

78.0% → 98.3%
predictions within 1 star

0.889 → 0.383
mean absolute error, in stars

7
training runs, one change at a time

1. Summary

The goal was to teach a small language model to read a restaurant review and predict the 1–5 star rating. Starting from Qwen2.5-0.5B, I fine-tuned with QLoRA and then ran controlled experiments, changing one setting at a time: adapter size, learning rate, which layers are adapted, number of epochs, amount of training data, and finally the base model size.

Main finding: tuning the adapter gave no gain; more training time and more data gave small gains; switching to a larger base model (Qwen2.5-1.5B) gave the biggest improvement. The best model reaches **65.0% accuracy**, **98.3% within one star** and an average error of **0.38 stars** on 300 unseen reviews.

2. Task and data

- **Task:** 5-class classification phrased as text generation: the model writes the star digit after the prompt.
- **Dataset:** `Yelp/yelp_review_full` (650,000 training and 50,000 test reviews, balanced across the five ratings).
- **Splits** (all shuffled with seed 42):

Split	Source	Size	Used for
Training	Yelp train split	2,000 (Run 1–D) / 5,000 (E, F)	Learning
Validation	Yelp train split, reviews #2000–2199	200	Validation loss, overfitting check
Test	Yelp test split	300	Accuracy, within-1-star, MAE

The 5,000-review set uses reviews #0–1999 and #2200–5199, so it never overlaps the validation set (checked in code: overlap = 0). Every model is evaluated on the **same 300 test reviews**.

3. Method

Prompt → completion	`Review: <first 600 characters>\n\nHow many stars (1–5) did the reviewer give?\nStars:` → `4`
Quantization	4-bit NF4 with double quantization (bitsandbytes); compute in float32, because the T4 has no real bf16 support

LoRA (baseline)	r=16, alpha=32, dropout 0.05 on q_proj, k_proj, v_proj, o_proj: 2.2M trainable parameters on 0.5B (0.44%), 4.4M on 1.5B (0.28%)
Training	TRL SFTTrainer, loss on the completion only; batch 4 × gradient accumulation 4 = 16; lr 2e-4, cosine schedule, 3% warmup; paged_adamw_32bit; max length 192 tokens; seed 42
Evaluation	Greedy generation of up to 3 tokens; the first digit 1–5 is the prediction (an unparseable answer counts as 3)
Safety of results	Every adapter and its results.json are pushed to Hugging Face right after training, before testing

Metrics. *Accuracy*: exact star correct. *Within 1 star*: at most one star off. *MAE*: mean absolute difference between predicted and true stars. *Validation loss*: cross-entropy on the 200 validation reviews (completion tokens only).

4. Results

Model	Base	Train reviews	Epochs	Accuracy ↑	Within 1 star ↑	MAE ↓	Val loss ↓
No fine-tuning	0.5B	–	–	0.427	0.780	0.889	–
Run 1: r=16, lr=2e-4, attention	0.5B	2,000	1	0.590	0.957	0.467	–
Exp A: bigger adapter (r=32)	0.5B	2,000	1	0.590	0.943	0.487	0.337
Exp B: smaller lr (1e-4)	0.5B	2,000	1	0.580	0.950	0.497	0.329
Exp C: attention + MLP	0.5B	2,000	1	0.573	0.947	0.490	0.320
Exp D: 2 epochs	0.5B	2,000	2	0.597	0.960	0.453	0.318
Exp E: 5,000 reviews	0.5B	5,000	2	0.607	0.967	0.440	0.302
No fine-tuning	1.5B	–	–	0.560	0.927	0.547	–
Exp F: Qwen2.5-1.5B	**1.5B**	**5,000**	**2**	**0.650**	**0.983**	**0.383**	**0.271**

Run 1 was trained in an earlier notebook with a slightly different setup; its validation loss was measured differently and is not comparable, so it is omitted.

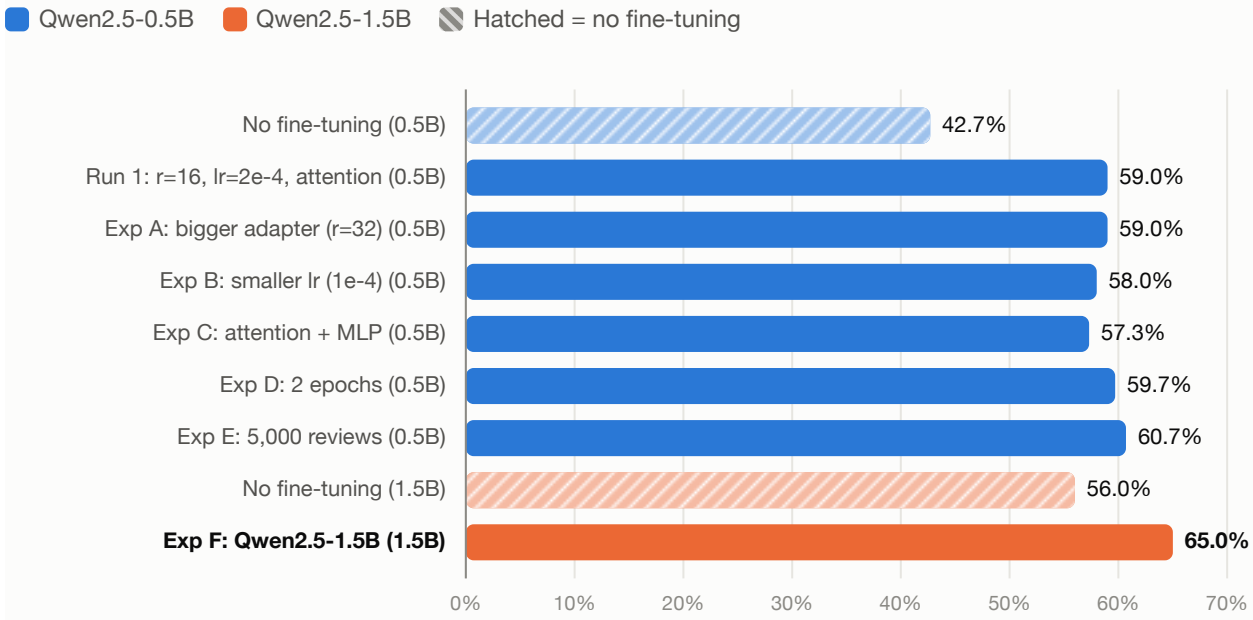


Figure 1. Exact-star accuracy on the 300 test reviews. Hatched bars are the base models before fine-tuning.

5. Experiments one by one

Experiment A: bigger adapter (r=32, alpha=64)

Doubling the adapter (4.3M parameters) left accuracy unchanged (0.590) and slightly worsened within-1-star and MAE. With 2,000 reviews, the adapter's capacity was not the bottleneck. Keeping r=16 is simpler and just as good.

Experiment B: smaller learning rate (1e-4)

Training loss fell more slowly at every checkpoint, and all three test metrics were slightly worse (0.580). With only 125 steps, half-size steps left the model under-trained.

Experiment C: attention + MLP layers

Adapting the MLP layers too (8.8M parameters, 4× more) gave the lowest validation loss of the 1-epoch runs (0.320) but the lowest accuracy (0.573). This shows that a lower loss does not guarantee more correct answers.

Experiment D: 2 epochs

Seeing each review twice improved every metric over Run 1 (0.597 / 0.960 / 0.453). Validation loss flattened after about 200 steps, suggesting 2 epochs is close to the useful limit for 2,000 reviews.

Experiment E: 5,000 training reviews

With D's settings and 2.5× more data, every metric improved again (0.607 / 0.967 / 0.440) and validation loss kept falling through epoch 2 (best 0.302). More data also kept the train–validation gap slightly smaller than in D.

Experiment F: Qwen2.5-1.5B

The untrained 1.5B model already scored 0.560, versus 0.427 for the untrained 0.5B. Fine-tuned with E's exact settings (89 minutes on a T4), it reached **0.650 / 0.983 / 0.383** and the lowest validation loss of the project (0.271). The +4.3-point gain over E (about 13 more correct reviews out of 300) is the only gain in this project that is clearly larger than test-set noise.

6. Learning curves and overfitting check

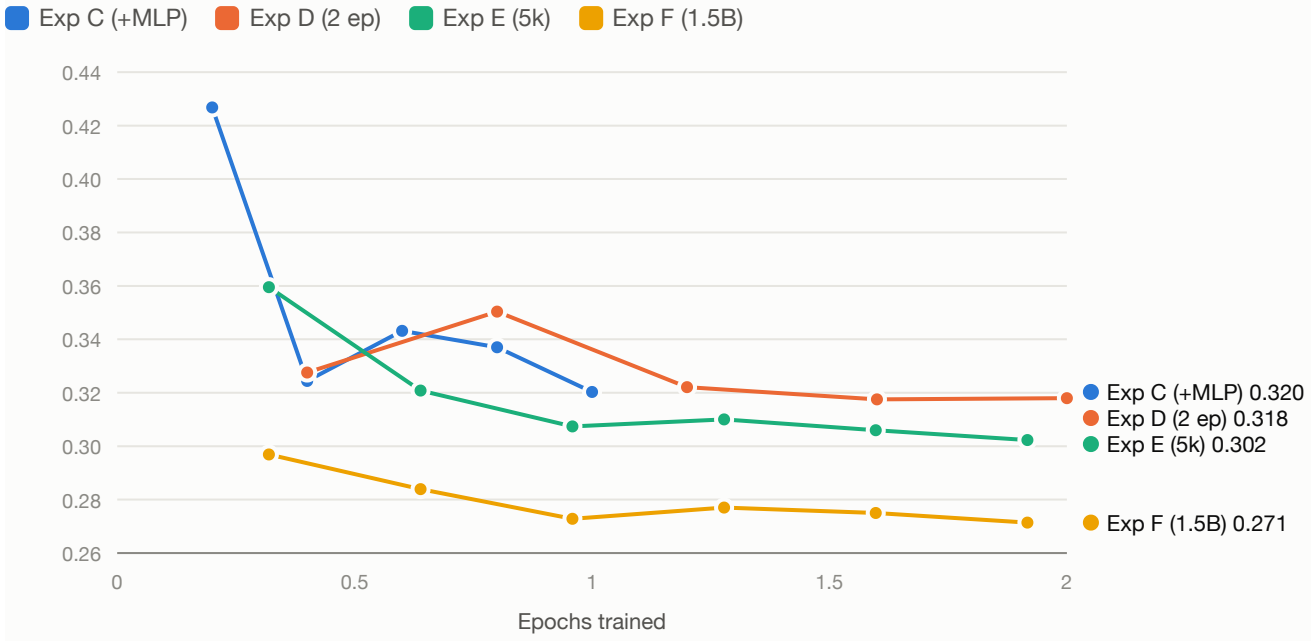


Figure 2. Validation loss during training, aligned by epochs. The 1.5B model (F) is lower at every checkpoint; more data (E) keeps improving where D flattens.

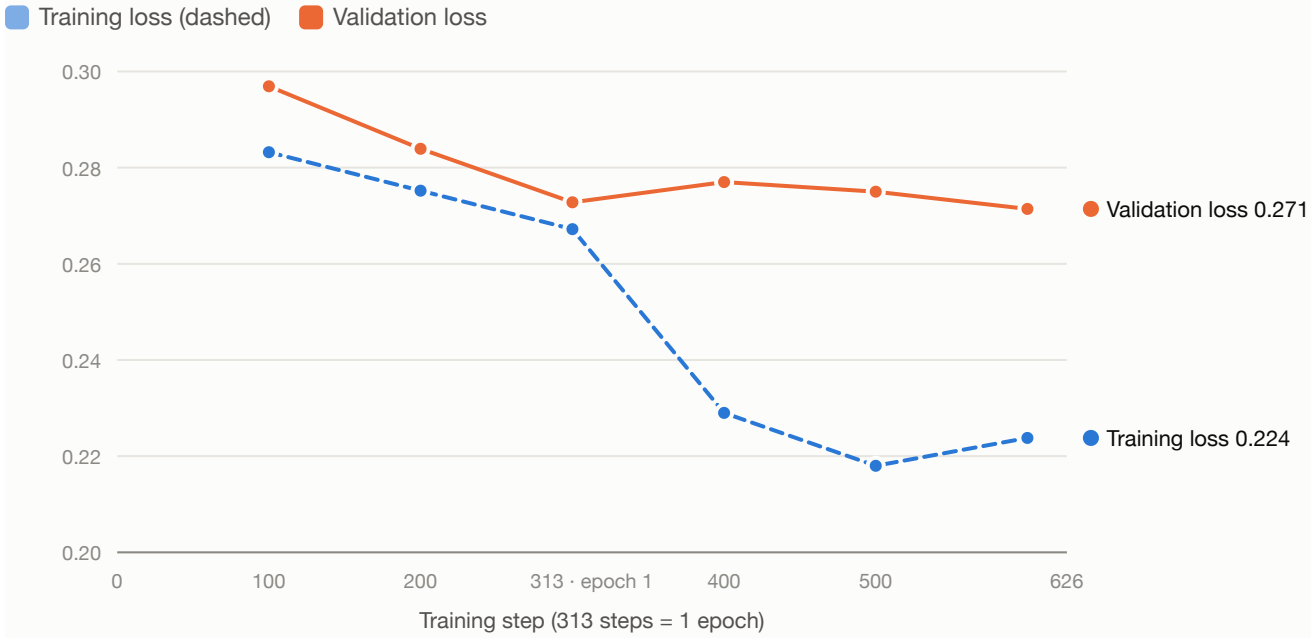


Figure 3. Experiment F, training vs validation loss. In epoch 2 training loss keeps falling while validation loss flattens: the gap grows from about 0.006 to 0.047, an early sign of overfitting, but validation loss never rises, so the final model is not overfit.

> **Rule used:** overfitting = training loss going down while validation loss goes up for several checkpoints in a row. A growing gap alone is a warning, not a failure. No run met the rule. Because validation loss flattened in epoch 2 for D, E and F, a third epoch would most likely add cost without improving the model.

7. Error analysis (Run 1)

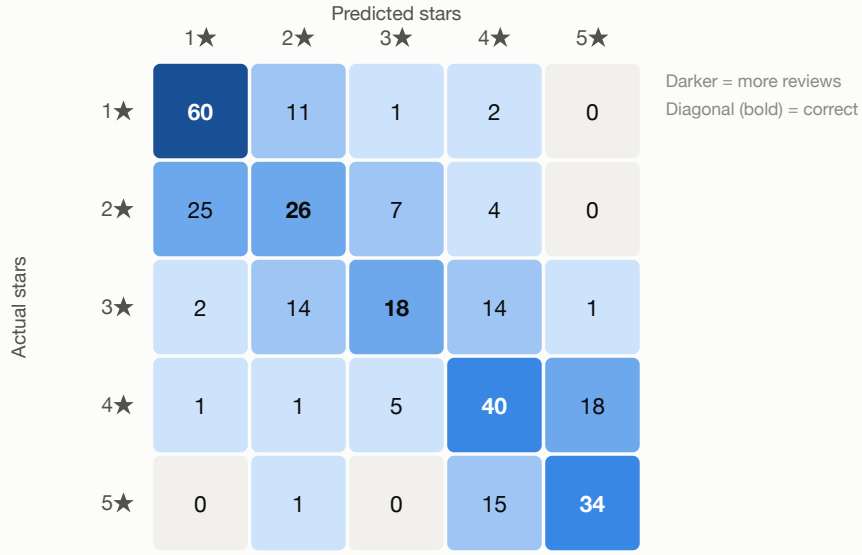


Figure 4. Confusion matrix for Run 1 (0.5B, 300 test reviews). 122 reviews were wrong, and almost all of them by one star.

- **Strongest at the extremes:** 60 of 74 one-star reviews were correct.
- **Weakest on mixed reviews:** only 18 of 49 three-star reviews were correct; they spread evenly to 2 and 4.
- **4 vs 5 is hard:** 18 four-star reviews were predicted as 5, and 15 five-star reviews as 4.
- These are cases where human raters also disagree. Published state-of-the-art models on Yelp 5-class reach roughly 70–73% accuracy, so 65% from a 1.5B model trained on 5,000 reviews is a reasonable result.

8. Lessons learned

1. **Change one thing at a time.** Each experiment changed exactly one setting against the same data, seed and test set, so every difference has a clear cause.
2. **Bigger adapters are not automatically better.** r=32 and MLP layers added parameters without adding information.
3. **Data and base-model quality matter most.** More data helped a little; a smarter base model helped a lot.
4. **Loss ≠ accuracy.** Always confirm with the real task metric.
5. **Watch both losses** to catch overfitting early and to choose the number of epochs.
6. **Save before evaluating.** An adapter was lost once to a runtime reset; afterwards every adapter was uploaded to Hugging Face immediately after training.

9. Limitations

- **Small test set.** With 300 reviews, differences of 1–2 points (3–6 reviews) are within noise. Only the gains from fine-tuning and from the 1.5B model are clearly significant.
- **Single seed.** Each configuration was trained once; repeated runs would give error bars.
- **Model selection on the test set.** Experiments were compared on test accuracy; strictly, choices should be made on validation data and the test set used once.
- **Truncation.** Reviews are cut at 600 characters, which can remove a conclusion at the end of a long review.

- **Run 1** was trained in a different notebook, so its loss is not comparable with the later runs.

10. Next steps

- 10,000–20,000 training reviews and longer review text (1,200 characters)
- Score the five answers by probability and predict the expected star value, to lower MAE without retraining
- A 1,000+ review test set and 2–3 seeds per run for statistically clearer comparisons
- A live Gradio demo on Hugging Face Spaces using the Experiment F adapter

11. Reproducibility

Code	github.com/Shiva123-del/review-star-rater-qlora (notebooks with outputs)
Best adapter	huggingface.co/shivkrchy9876/review-star-rater-exp-f-1.5b (with results.json and final_comparison.json)
Other adapters	review-star-rater-qlora (Run 1), -exp-a-r32, -exp-b-lr1e-4, -exp-c-mlp, -exp-d-2epochs, -exp-e-5k
Libraries	transformers 4.57.6, trl 0.25.1, bitsandbytes 0.48.2, peft, datasets, accelerate
Hardware	Kaggle T4 (16 GB): 6.1 min (A, B), 8.2 min (C), 12.7 min (D), 30.3 min (E), 88.7 min (F)

Appendix: loss tables

Experiment C: validation loss

Step	25	50	75	100	125
Val loss	0.427	0.324	0.343	0.337	0.320

Experiment D: validation loss

Step	50	100	150	200	250
Val loss	0.328	0.350	0.322	0.318	0.318

Experiment E: validation loss

Step	100	200	300	400	500	600
Val loss	0.359	0.321	0.307	0.310	0.306	0.302

Experiment F: validation loss

Step	100	200	300	400	500	600
Val loss	0.297	0.284	0.273	0.277	0.275	0.271

Experiment F: training loss

Step	100	200	300	400	500	600
Train loss	0.283	0.275	0.267	0.229	0.218	0.224